

ADVANCED PROGRAMMING LAB REPORT

Assignment 18

Name	Aritra Talukdar
Roll Number	CSB24034
Department	Computer Science and Engineering
Subject	Advanced Programming
Assignment Topic	Exception Handling and File Processing in Python

School of Engineering
Tezpur University

May 19, 2026

1. Aim

The aim of this assignment is to implement a Python program that reads a numeric score from a text file, processes the score by multiplying it by 10, and demonstrates robust exception handling using Python's `try-except-else-finally` structure. The assignment also aims to validate the program functionality using automated unit testing with the `pytest` framework.

2. Objectives

- To understand file handling in Python.
- To understand exception handling mechanisms.
- To implement error handling for missing files and invalid data formats.
- To learn the usage of `try`, `except`, `else`, and `finally` blocks.
- To understand automated testing using `pytest`.
- To validate both successful and erroneous execution cases.

3. Theory

3.1. File Handling in Python

File handling is the process of reading from and writing to files. Python provides built-in functions for opening, reading, and closing files.

The `open()` function is used to open a file.

```
1 with open("file.txt", "r") as file:
```

The `"r"` mode indicates read mode.

The `with` statement automatically closes the file after usage, ensuring proper resource management.

3.2. Exception Handling

Exception handling is used to prevent abrupt program termination during runtime errors.

Python provides:

- `try` block
- `except` block
- `else` block
- `finally` block

3.3. FileNotFoundError

This exception occurs when the specified file does not exist.

Example:

```
1 FileNotFoundError
```

3.4. ValueError

This exception occurs when invalid data is converted into another datatype.

Example:

```
1 int("hello")
```

This generates:

```
1 ValueError
```

3.5. Pytest Framework

Pytest is a testing framework used for automated testing in Python.

It helps verify whether:

- The program produces correct outputs.
- Exceptions are handled properly.
- Program logic works correctly.

4. Algorithm

1. Start the program.
2. Open the input file using `open()`.
3. Read the contents of the file.
4. Remove extra spaces or newline characters using `strip()`.
5. Convert the string into an integer using `int()`.
6. Multiply the integer by 10.
7. Return the processed result.
8. If file is missing, handle `FileNotFoundError`.
9. If invalid data is present, handle `ValueError`.
10. Print cleanup message in the `finally` block.
11. Stop the program.

5. Program

5.1. Main Python Program

```
1
2 # =====
3 # File Name : score_processor.py
4 # Purpose   : Read a score from a file,
5 #             multiply it by 10,
6 #             and handle exceptions properly.
7 # =====
8
9
10 # -----
11 # Creating a class named ScoreProcessor
12 # -----
13 class ScoreProcessor:
14
15     # -----
16     # Method : process_score_file
17     #
18     # Parameter:
19     #   file_path -> path of the text file
20     #
21     # Returns:
22     #   Integer result after multiplication
23     # -----
24     def process_score_file(self, file_path: str) -> int:
25
26         # -----
27         # try block:
28         # Code that may generate exceptions
29         # -----
30         try:
31
32             # -----
33             # Open file in read mode
34             #
35             # "with" automatically closes the file
36             # after use.
37             # -----
```

```
38         with open(file_path, "r") as file:
39
40             # -----
41             # Read data from file
42             # -----
43             score_text = file.read()
44
45             # -----
46             # Remove extra spaces/newline characters
47             # -----
48             score_text = score_text.strip()
49
50             # -----
51             # Convert text into integer
52             #
53             # Possible Error:
54             # ValueError if data is not numeric
55             # -----
56             score = int(score_text)
57
58             # -----
59             # Multiply score by 10
60             # -----
61             result = score * 10
62
63             # -----
64             # Handles missing file error
65             # -----
66         except FileNotFoundError:
67
68             print("Error: File not found.")
69
70             # Re-raise exception for pytest
71             raise
72
73             # -----
74             # Handles invalid integer conversion
75             # -----
76         except ValueError:
77
78             print("Error: Invalid number format in file.")
79
80             # Re-raise exception for pytest
```

```

81         raise
82
83     # -----
84     # Executes ONLY if no exception occurred
85     # -----
86     else:
87
88         print("Data processed successfully")
89
90         return result
91
92     # -----
93     # Executes ALWAYS
94     # -----
95     finally:
96
97         print("File cleanup completed")
98
99
100 # =====
101 # Main Section
102 # This code runs only when this file is executed directly
103 # =====
104 if __name__ == "__main__":
105
106     # Create object of ScoreProcessor
107     processor = ScoreProcessor()
108
109     # Call method using valid file
110     final_result = processor.process_score_file("valid_score.txt"
111 )
112
113     # Display final result
114     print("Final Result =", final_result)

```

5.2. Pytest Test Program

```

1
2 # =====
3 # File Name : test_score_processor.py
4 # Purpose   : Unit tests for ScoreProcessor

```

```
5 # =====
6
7
8 # -----
9 # Import pytest framework
10 # -----
11 import pytest
12
13 # -----
14 # Import ScoreProcessor class
15 # -----
16 from score_processor import ScoreProcessor
17
18
19 # -----
20 # Test Case 1
21 # Test successful score processing
22 # -----
23 def test_valid_score_processing():
24
25     # Create object
26     processor = ScoreProcessor()
27
28     # Call method using valid file
29     result = processor.process_score_file("valid_score.txt")
30
31     # Verify expected output
32     # 8 * 10 = 80
33     assert result == 80
34
35
36 # -----
37 # Test Case 2
38 # Test missing file handling
39 # -----
40 def test_missing_file():
41
42     # Create object
43     processor = ScoreProcessor()
44
45     # Expect FileNotFoundError
46     with pytest.raises(FileNotFoundError):
47
```

```
48     processor.process_score_file("missing_file.txt")
49
50
51 # -----
52 # Test Case 3
53 # Test invalid number format
54 # -----
55 def test_invalid_data():
56
57     # Create object
58     processor = ScoreProcessor()
59
60     # Expect ValueError
61     with pytest.raises(ValueError):
62
63         processor.process_score_file("invalid_score.txt")
```

6. Input Files

6.1. Valid Input File

Filename: valid_score.txt

```
1
2 8
```

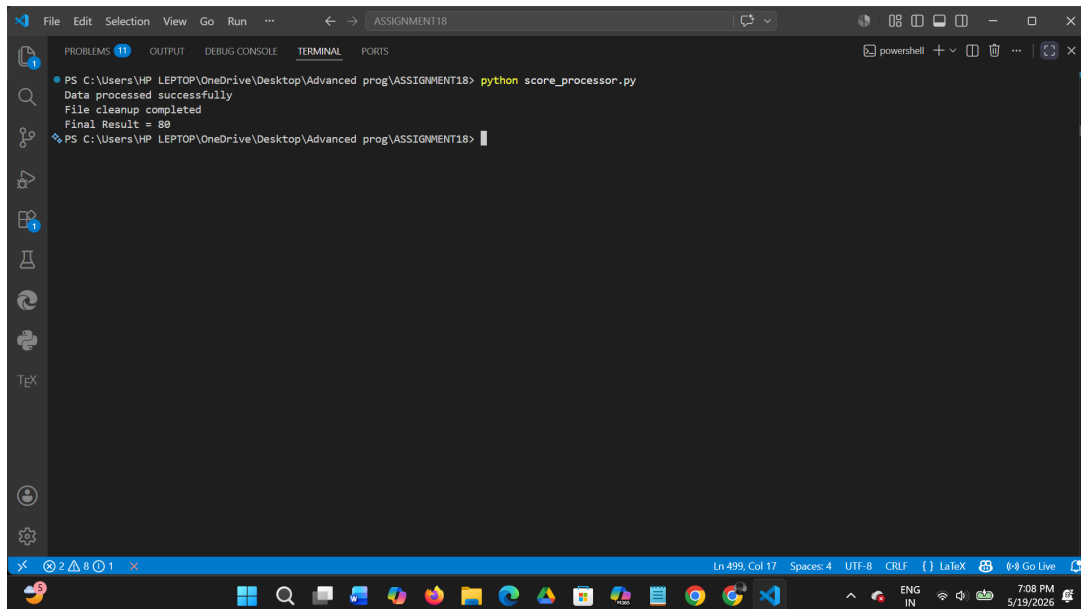
6.2. Invalid Input File

Filename: invalid_score.txt

```
1
2 hello!
```


7. Outputs

7.1. Output Screenshot for Successful Execution

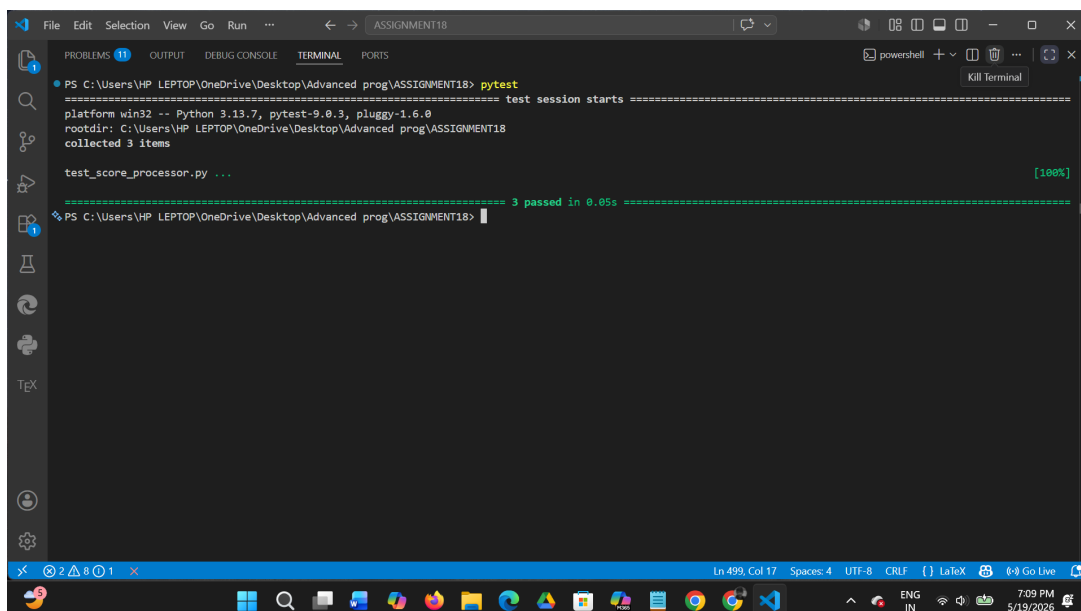


The screenshot shows a terminal window with the following output:

```
PS C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT18> python score_processor.py
Data processed successfully
File cleanup completed
Final Result = 80
PS C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT18>
```

Figure 1: Successful Program Execution

7.2. Output Screenshot for Pytest Execution



The screenshot shows a terminal window with the following output:

```
PS C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT18> pytest
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT18
collected 3 items

test_score_processor.py ... [100%]

===== 3 passed in 0.05s =====
PS C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\ASSIGNMENT18>
```

Figure 2: Pytest Execution Output

8. Result

The Python program was successfully implemented to read a numeric score from a text file, multiply it by 10, and handle runtime exceptions properly. The program successfully handled missing file errors and invalid data formats using exception handling techniques. Automated testing using the `pytest` framework verified both successful and erroneous execution scenarios.

9. Conclusion

This assignment helped in understanding the practical implementation of file handling, exception handling, and automated testing in Python. The usage of `try-except-else-finally` blocks ensured robust error handling and resource management. The implementation of `pytest` demonstrated the importance of automated testing in software development.